The Journal of Open Source Software

KrakenParser: Post-processing of Kraken2-like Taxonomic Reports

Ilia V. Popov [1]

1 Faculty of Bioengineering and Veterinary Medicine, Don State Technical University, Russia

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Claudia Solis-Lemus

Reviewers:

- @Ruslan-Kurmashev
- @KristinaGagalova

Submitted: 01 June 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

KrakenParser is an open-source Python library and command-line interface (CLI) designed to automate and standardize the post-processing of metagenomic taxonomic classification reports generated by Kraken2 (Wood et al., 2019) and Bracken (Lu et al., 2017), which reports its re-estimated abundances in the same Kraken2-style report format. Although these high-throughput profilers provide rapid taxonomic assignments, their primary outputs comprise independent, hierarchical text files that are challenging to aggregate, filter, and analyze across multi-sample cohorts. KrakenParser converts these Kraken2-compatible taxonomic reports into unified, rank-specific (from phylum down to species) count tables formatted as tidy CSVs, applies taxonomic filtering and relative-abundance normalization, calculates ecological alpha and beta diversity metrics, and produces publication-ready visualizations for bioinformaticians and microbiome researchers. KrakenParser can be deployed as an automated end-to-end pipeline via a single CLI command or integrated as a modular library within custom bioinformatics workflows.

Statement of need

Comparative metagenomics and microbiome studies depend fundamentally on cross-sample data matrices to perform downstream statistical profiling and ecological modeling. Existing utilities only partially address this requirement. The KrakenTools suite (Lu et al., 2022) provides low-level text manipulation scripts but lacks an integrated, automated execution framework, requiring manual multi-step invocations and custom scripting to generate structured matrices. Graphical platforms like Pavian (Breitwieser & Salzberg, 2020) excel at interactive data exploration but are poorly suited for headless batch execution, high-performance computing (HPC) clusters, or continuous integration pipelines. Lightweight command-line utilities, such as spideog (Manguy, 2020), convert one or several reports to flat CSV or Newick-tree files but do not provide relative-abundance normalization, diversity calculations, or plotting capabilities.

KrakenParser consolidates these fragmented workflows into a single environment. Since the inception of fast k-mer based classification (Wood & Salzberg, 2014), the taxonomic profiling ecosystem has evolved to include diverse specialized algorithms such as Centrifuge (Kim et al., 2016), CLARK (Ounit et al., 2015), Kaiju (Menzel et al., 2016), KrakenUniq (Breitwieser et al., 2018), and standardized benchmarking standards like CAMI (Sczyrba et al., 2017). However, the lack of standardized downstream processing tools remains a barrier. As an ideological successor to KrakenTools (Lu et al., 2022), KrakenParser abstracts low-level file parsing into a robust pipeline, allowing researchers to transition from raw outputs to ecological indices and visualizations without intermediate data-wrangling scripts, thereby ensuring reproducibility across cohort studies.

DRAFT

Comparison with related software

The table below summarizes, based on the official documentation of each project, how `KrakenParser` relates to `KrakenTools` (Lu et al., 2022), `Pavian` (Breitwieser & Salzberg, 2020), and `spideog` (Manguy, 2020) along the axes of supported input, batch processing, diversity calculations, visualization, and mode of use.

Feature	`KrakenParser`	`KrakenTools`	`Pavian`	`spideog`
Supported input	Kraken2 reports and Bracken abundance-estimation output (Kraken2-style format)	Kraken 1, Kraken 2, KrakenUniq, and Bracken reports; several scripts also accept raw FASTA/FASTQ read files	Kraken, KrakenUniq, Kraken 2, Centrifuge, and MetaPhlAn-style reports, plus BAM alignments for its viewer	Kraken2 and Bracken reports
Batch processing	Single CLI invocation ingests a directory of reports and produces rank-specific cross-sample matrices	`combine_kreports.py` & `combine_mpa.py` merge multiple reports, but each script is invoked and chained by the user; no unified multi-sample driver	Multi-sample comparison view within the interactive browser session; no scriptable or headless batch mode	`combine-abundances` / `combine-trees` subcommands merge multiple reports into one CSV or Newick file per invocation
Diversity calculations	Built-in alpha (Shannon, Pielou, Chao1, computed on raw counts) and beta (Bray-Curtis, Jaccard, with rarefaction) diversity	`DiversityTools` add-on scripts: alpha (Shannon, Berger-Parker, Simpson, inverse Simpson, Fisher), one sample at a time, and beta (Bray-Curtis only); neither includes rarefaction	None	None
Visualization	Stacked bar plots, streamgraphs, and clustermaps rendered as static figures	None built in; `kreport2krona.py` exports a Krona-compatible text file for rendering with the external Krona tool	Interactive Sankey diagrams, sample overview plots, and a BAM alignment pileup viewer rendered in the browser	None

DRAFT

Feature	KrakenParser	KrakenTools	Pavian	spideog
Mode of use	Single CLI command for the full pipeline, or modular subcommands, suited to scripted/headless execution (HPC, CI)	Suite of independent Python CLI scripts, each run and chained manually by the user	Interactive R/Shiny web application (local install or Docker), designed for exploratory use rather than headless automation	Standalone compiled CLI binary performing one format-conversion operation per invocation

Implementation and Software Design

KrakenParser is implemented in Python 3 (distributed via PyPI as krakenparser) and follows a modular architecture split into three distinct operational layers: Data Processing, Statistical Analysis, and Visualization. The pipeline can be executed in an end-to-end automated mode by providing global input and output paths directly to the main command, or controlled step-by-step through granular subcommands.

Data Processing and Filtering

Individual taxonomic reports are programmatically parsed, converted into MetaPhlAn (MPA) format tables (Segata et al., 2012), and merged into a unified cross-sample master count matrix. This matrix is subsequently deconstructed into distinct tables for each major taxonomic rank. During deconstruction, KrakenParser purges internal structural prefixes (e.g., stripping s__ from species names) and normalizes taxonomic strings by replacing underscores with spaces to ensure human readability and compatibility with downstream software.

The core data engine features flexible filtering mechanisms. Users can selectively isolate or exclude specific biological domains or kingdoms (Bacteria, Viruses, Archaea, Fungi) during extraction. While non-target host reads (e.g., human contamination) are filtered out by default to focus on microbial signatures, the --keep-human flag preserves host read counts within the output matrices. Crucially, --keep-human can be combined concurrently with domain-specific filters, allowing the simultaneous evaluation of host-to-microbe or host-to-pathogen abundance ratios within a single run.

Statistical Analysis

Following matrix generation, the statistical module computes normalization metrics and ecological indices directly:

- **Relative Abundance:** Normalizes absolute counts into percentage distributions using the formula: $\text{Relative Abundance} = \left(\frac{\text{Number of individuals of taxa}}{\text{Total number of individuals of all taxa}}\right) \times 100$. A user-defined abundance threshold aggregates rare background taxa into a consolidated Other category to simplify downstream parsing and plotting.
- **Alpha Diversity:** Calculates *Shannon* (Shannon, 1948), *Pielou's evenness* (Pielou, 1966), and *Chao1* (Chao & Bunge, 2002) indices directly from the unrarefied input count matrix.
- **Beta Diversity:** Computes compositional dissimilarity between samples via *Bray-Curtis* (Bray & Curtis, 1957) and *Jaccard* (Jaccard, 1901) distance metrics using NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020), applying a user-defined rarefaction depth prior to matrix calculation to account for uneven sequencing depths, exporting standard distance matrices ready for ordination.

DRAFT

Visualization

The `kpplot` module utilizes an object-oriented design inheriting from a unified base configuration class (`KpPlotBase`), enforcing consistent rendering properties such as DPI, bounding box scaling, and layout properties. Built on top of `matplotlib` (Hunter, 2007), `pandas` (McKinney, 2010), and `seaborn` (Waskom, 2021), the visualization engine exposes three primary programmatic layouts:

- **Stacked Bar Plots:** For comparing relative taxonomic proportions across multi-sample cohorts.
- **Streamgraphs:** Using symmetric baselines to map continuous structural transitions or temporal trends in taxonomic composition.
- **Clustermaps:** Applying unsupervised hierarchical clustering to reveal sample similarities and taxonomic co-occurrence blocks.

Research Impact

The functional reliability and execution integrity of `KrakenParser` are validated via automated continuous integration workflows. The utility and user readiness of the software were demonstrated during the 2025 "Bioinformatics Bootcamp" hackathon. Furthermore, the core structural prototype of this tool was successfully utilized for large-scale metagenomic data analysis by Popov et al., 2025 (Popov et al., 2025).

AI usage disclosure

Generative AI tools were used during the development of this work to assist with code refactoring, documentation drafting, and manuscript text editing. All software design decisions, implementation, validation, and scientific interpretation were performed and reviewed by the authors. No generative AI tools were used to generate or analyze research data, and all results reported are reproducible from the publicly available source code and documentation.

References

Bray, J. R., & Curtis, J. T. (1957). An ordination of the upland forest communities of southern wisconsin. *Ecological Monographs*, *27*(4), 325–349. https://doi.org/10.2307/1942268

Breitwieser, F. P., Baker, D. N., & Salzberg, S. L. (2018). KrakenUniq: Confident and fast metagenomics classification using unique k-mer counts. *Genome Biology*, *19*, 198. https://doi.org/10.1186/s13059-018-1568-0

Breitwieser, F. P., & Salzberg, S. L. (2020). Pavian: Interactive analysis of metagenomics data for microbiome studies and pathogen identification. *Bioinformatics*, *36*(4), 1303–1304. https://doi.org/10.1093/bioinformatics/btz715

Chao, A., & Bunge, J. (2002). Estimating the number of species in a stochastic abundance model. *Biometrics*, *58*(3), 531–539.

Harris, C. R., Millman, K. J., Walt, S. J. van der, & others. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des alpes et des jura. *Bull Soc Vaudoise Sci Nat*, *37*, 547–579.

Kim, D., Song, L., Breitwieser, F. P., & Salzberg, S. L. (2016). Centrifuge: Rapid and

DRAFT

sensitive classification of metagenomic sequences. *Genome Research*, *26*(12), 1721–1729. https://doi.org/10.1101/gr.210641.116

Lu, J., Breitwieser, F. P., Thielen, P., & Salzberg, S. L. (2017). Bracken: Estimating species abundance in metagenomics data. *PeerJ Computer Science*, *3*, e104. https://doi.org/10.7717/peerj-cs.104

Lu, J., Rincon, N., Wood, D. E., Breitwieser, F. P., Pockrandt, C., Langmead, B., Salzberg, S. L., & Steinegger, M. (2022). Metagenome analysis using the kraken software suite. *Nature Protocols*, *17*, 2815–2839. https://doi.org/10.1038/s41596-022-00738-y

Manguy, J. (2020). *Spideog: Command line utility for Kraken2 reports*. GitHub. https://github.com/jeanmanguy/spideog

McKinney, W. (2010). Data structures for statistical computing in python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th python in science conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Menzel, P., Ng, K. L., & Krogh, A. (2016). Fast and sensitive taxonomic classification for metagenomics with kaiju. *Nature Communications*, *7*, 11257. https://doi.org/10.1038/ncomms11257

Ounit, R., Wanamaker, S., Close, T. J., & Lonardi, S. (2015). CLARK: Fast and accurate classification of metagenomic and genomic sequences using discriminative k-mers. *BMC Genomics*, *16*, 236. https://doi.org/10.1186/s12864-015-1419-2

Pielou, E. C. (1966). The measurement of diversity in different types of biological collections. *Journal of Theoretical Biology*, *13*, 131–144.

Popov, I. V., Manakhov, A. D., Gorobets, V. E., Diakova, K. B., Lukbanova, E. A., Malinovkin, A. V., Venema, K., Ermakov, A. M., & Popov, I. V. (2025). Metagenomic investigation of intestinal microbiota of insectivorous synanthropic bats: Densoviruses, antibiotic resistance genes, and functional profiling of gut microbial communities. *International Journal of Molecular Sciences*, *26*(13). https://doi.org/10.3390/ijms26135941

Sczyrba, A., Hofmann, P., Belmann, P., Koslicki, D., Janssen, S., Dröge, J., Gregor, I., Majda, S., Fiedler, J., Dahms, E., & others. (2017). Critical assessment of metagenome interpretation: A benchmark of metagenomics software. *Nature Methods*, *14*(11), 1063–1071. https://doi.org/10.1038/nmeth.4458

Segata, N., Waldron, L., Ballarini, A., Narasimhan, V., Jousson, O., & Huttenhower, C. (2012). Metagenomic microbial community profiling using unique clade-specific marker genes. *Nature Methods*, *9*, 811–814. https://doi.org/10.1038/nmeth.2066

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, *27*(3), 379–423.

Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021. https://doi.org/10.21105/joss.03021

Wood, D. E., Lu, J., & Langmead, B. (2019). Improved metagenomic analysis with kraken 2. *Genome Biology*, *20*, 257. https://doi.org/10.1186/s13059-019-1891-0

Wood, D. E., & Salzberg, S. L. (2014). Kraken: Ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, *15*(3), R46. https://doi.org/10.1186/gb-2014-15-3-r46

DRAFT